

Rated

Software Dependability Report

Bruno Nesticò • Francesco Rao • Roberto Fiorenza

10 September 2026

github.com/superfra01/Rated-Dependability



Index

- 1 The Rated project: context, architecture, technologies
- 2 GitHub workflows: Continuous Integration and Continuous Delivery
- 3 Formal verification with OpenJML
- 4 Docker environment configuration
- 5 Testing activities: JaCoCo code coverage and PIT mutation testing
- 6 Performance testing: JMH and Creedengo static analysis
- 7 Security: native GitHub features, SonarQube, GitGuardian, Snyk
- 8 Final considerations

1

The Rated project

The Rated project

- ▶ **Domain:** web platform that enables users to rate and review movies and TV series
- ▶ **For users:** navigate through the catalog (movies and series), explore in detail every title presented in the catalog, see its data sheet, score its overall impression with stars, and compose a written review on it. Furthermore, a user is be able to rate, comment, and leave their opinion on other users' reviews, as well as on the reviews' replies. During browsing, users are able to also access the profiles of other users.
- ▶ **For administrators:** catalog management (insertion, modification and removal of titles) and moderation of comments alongside user account control.
- ▶ **Three-tier architecture:** clear separation between presentation layer, application logic and persistence management, guaranteeing maintainability and scalability.
 - **Frontend:** HTML5, CSS3, JavaScript
 - **Backend:** Java (Servlet and JSP), Tomcat 9
 - **Database:** MySQL, accessed through JDBC and a JNDI DataSource

Evaluation criteria and Results

Criterion	Result
Application buildable in CI/CD and locally	Ci.yml and continous delivery on every push/PR
Formal JML specification verified with OpenJML	27 annotated files and formal verification successful
Docker image ready to be orchestrated	docker-push.yml and dockerization for both amd64 and arm64
Significant number of test cases	361 tests in 94 classes (unit and integration)
Code coverage analyzed with JaCoCo	96% instructions and 93% branches (from 58% / 41%)
Mutation testing campaign with PIT	98% mutation coverage · 98% test strength
JMH microbenchmarks on demanding components	7 service suites · up to -22.7% ns/op
Security mechanisms implemented in CI/CD	GitGuardian, Snyk, SonarQube, Dependabot
Security analyzed with the three tools	97 → 0 issues; remaining findings = false positives
No vulnerabilities in the web application	Security Rating A · 0 security issues

2

GitHub workflows

Continuous Integration - ci.yml

- ▶ **Trigger:** every push and every pull request on the main branch.
- ▶ **Checkout and setup:** the repository is checked out and the correct JDK 17 is installed
- ▶ **Build and test:** the ***mvn verify*** stage performs a full build of the project, executes all the tests, and generates the code coverage report with JaCoCo.
- ▶ **Mutation testing:** PIT evaluates the ability of the current test suite to detect defects in the project's code.
- ▶ **Upload of the reports:** the JaCoCo coverage report and the PIT mutation testing report are published as artifacts.
- ▶ **Secret scanning:** GitGuardian scans the code for secrets, API keys, tokens, and credentials.
- ▶ **Dependency analysis:** Snyk performs SCA on the Maven dependencies on the main branch with a fairly strict severity threshold, and Snyk SAST scans the code
- ▶ **Static analysis:** SonarQube analyzes the code base and pushes the analysis to SonarCloud, in order to detect bugs, vulnerabilities, code smells, etc.
- ▶ **Containerization:** the Docker image is built and tagged with the current commit hash, for full traceability of the artifact.

Continuous Delivery - docker-push.yml

- ▶ **Trigger:** a push to the main branch (or eventually manual execution)
- ▶ **Checkout and JDK 17:** the Java environment setup of the CI workflow is replicated.
- ▶ **Builder setup:** a Docker Buildx multi-architecture builder is initialized.
- ▶ **Authentication:** the workflow authenticates to Docker Hub using the corresponding GitHub Secrets.
- ▶ **Build and push:** the multi-architecture image is built for both linux amd64 and arm64.

3

Formal verification with OpenJML

OpenJML: objectives and scope

- ▶ **What it is:** OpenJML is a framework for formal specification and verification of Java programs, supporting ESC and RAC.
- ▶ **OpenJML** provides JML specifications for part of the Java standard library, so application contracts are checked together with the available library models.
- ▶ **Scope:** about 27 Java files, providing a model specification for all the business logic (model, dao, service).

10 Model classes

FilmBean, FilmGenereBean, GenereBean, InteresseBean, PreferenzaBean, RecensioneBean, ReportBean, UtenteBean, ValutazioneBean, VistoBean

Annotated with class invariants and contracts for constructors, getters and setters.

10 DAO classes

FilmDAO, FilmGenereDAO, GenereDAO, InteresseDAO, PreferenzaDAO, RecensioneDAO, ReportDAO, UtenteDAO, ValutazioneDAO, VistoDAO

Preconditions on parameters, frame conditions, nullability and minimal properties of returned objects and collections.

5 Services + 2 Utilities

CatalogoService, RecensioniService, AuthenticationService, ModerationService, ProfileService — modular reasoning about domain objects and DAOs.

FieldValidator and PasswordUtility: validation and hashing contracts, implementations treated as trusted boundaries.

OpenJML application strategy

- ▶ **Approach:** We verified the code bottom-up: first the model classes, then the DAOs, then the services. Each class and method was annotated with contracts (what must hold before it runs, what it guarantees afterwards, and what must always be true of the object). OpenJML then proves the code respects them.
- ▶ **Model:** the contracts say when an object is valid (for example, a film cannot have a negative duration) and the constructors make it impossible to create an invalid one.
- ▶ **DAO:** the contracts say what a method needs in order to work, what it gives back, and whether it changes the database.
- ▶ **Service:** the same reasoning applied to operations such as catalog, reviews, authentication, moderation.

OpenJML verification results

- ▶ **Complete ESC verification:** all 27 annotated Java files analyzed with OpenJML and the verification terminates with exit code zero.
- ▶ **RAC compilation:** completed successfully on the same 27 files, producing instrumented class files for runtime assertion checking.
- ▶ **Conclusion:** the correctness of the core business logic is effectively guaranteed by formal verification, not by tests alone.

```
=====
RISULTATO ESC: verifica statica completata senza problemi.
Codice di uscita ESC: 0
=====
```

```
=====
RISULTATO RAC: strumentazione completata senza problemi.
Codice di uscita RAC: 0
Codice di uscita complessivo: 0
=====
```

4

Docker environment configuration

The whole stack in containers

- ▶ The application and its database are packaged as Docker containers and started together from a single Compose file.
- ▶ One identical, reproducible environment on every machine



Reproducible image

The application is compiled inside the image and only the runtime is shipped, so the build never depends on what is installed on the developer's machine.

Isolated services

Application server and database run as two separate containers on a private network, each with its own health check and restart policy.

Built by the pipeline

The delivery workflow builds the image for both amd64 and arm64 and publishes it to Docker Hub on every push to main.

Inside the image: build and runtime

- ▶ The image is built in two stages: the application is compiled in the first, and only what is needed to run it is carried into the second so the compiler and the build tools never end up in the shipped image.

STAGE 1 • BUILD

- ▶ Maven 3.9 compiles the application and packages it into a single deployable file.
- ▶ The dependencies are downloaded before the sources are copied, so they are re-used from cache as long as the dependency list doesn't change.
- ▶ Nothing needs to be installed on the developer's machine: no JDK, no Maven.

rated-app.war



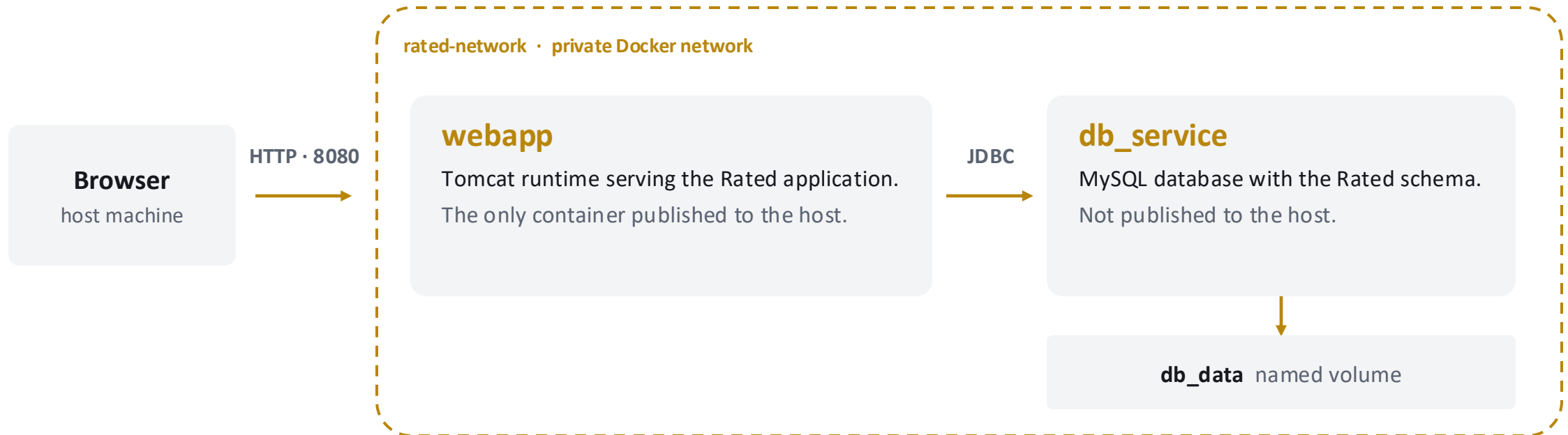
STAGE 2 • RUNTIME

- ▶ Only the packaged application crosses over from the first stage.
- ▶ Tomcat's default applications are removed, so the image contains nothing but what the project actually needs.
- ▶ Nothing needs to be installed on the developer's machine: no JDK, no Maven.

The database credentials are never written in the code or baked into the image but they are injected when the container starts.

Deployment architecture

- ▶ Only the application is reachable from the host, the database is not.



5

Testing activities

Code coverage with JaCoCo

Threshold set at 80% code and branch coverage

BEFORE • 58% instructions • 41% branches • 857 missed lines out of 1,992

Rated

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
sottosistemi.Gestione_Utenti.view		0%		0%	84	84	298	298	43	43	12	12
sottosistemi.Gestione_Catalogo.view		0%		0%	53	53	172	172	31	31	8	8
model.DAO		84%		71%	46	121	173	839	16	69	0	10
sottosistemi.Gestione_Recensioni.view		0%		0%	24	24	77	77	20	20	5	5
model.Entity		76%		n/a	9	102	55	245	9	102	0	10
model		0%		0%	10	10	39	39	8	8	2	2
utilities		43%		0%	9	10	13	21	6	7	1	2
sottosistemi.Gestione_Recensioni.service		88%		85%	5	26	10	105	1	12	0	1
sottosistemi.Gestione_Utenti.service		90%		90%	6	42	15	126	3	27	0	3
sottosistemi.Gestione_Catalogo.service		93%		100%	1	17	5	70	1	14	0	1
Total	2.774 of 6.691	58%	181 of 312	41%	247	489	857	1.992	138	333	28	54

Code coverage with JaCoCo

Threshold set at 80% code and branch coverage

AFTER • 96% instructions (298 missed of 8,510) • 93% branches (37 of 572)

Rated

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
 sottosistemi.Gestione_Utenti.view		92%		90%	19	142	46	559	2	56	0	12
 sottosistemi.Gestione_Catalogo.view		94%		97%	4	98	27	376	1	39	0	8
 model.DAO		98%		91%	6	97	11	844	1	69	0	10
 sottosistemi.Gestione_Recensioni.view		95%		100%	0	63	12	205	0	20	0	5
 sottosistemi.Gestione_Recensioni.service		98%		93%	4	41	0	117	0	12	0	1
 sottosistemi.Gestione_Utenti.service		98%		90%	5	52	1	143	0	27	0	3
 model.Entity		100%		n/a	0	102	0	245	0	102	0	10
 sottosistemi.Gestione_Catalogo.service		100%		94%	1	23	0	76	0	14	0	1
 utilities		100%		100%	0	10	0	21	0	7	0	2
 model		100%		75%	2	7	0	24	0	3	0	1
Total	298 of 8,510	96%	37 of 572	93%	41	635	97	2,610	4	349	0	53

Mutation testing with PIT

BEFORE

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
54	59% 1185/2012	47% 369/784	75% 369/494

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
model	2	0% 0/39	0% 0/8	100% 0/0
model.DAO	10	84% 716/855	58% 194/332	68% 194/284
model.Entity	10	78% 190/245	86% 37/43	86% 37/43
sottosistemi.Gestione_Catalogo.service	1	93% 65/70	50% 19/38	50% 19/38
sottosistemi.Gestione_Catalogo.view	8	0% 0/174	0% 0/57	100% 0/0
sottosistemi.Gestione_Recensioni.service	1	90% 95/105	84% 59/70	95% 59/62
sottosistemi.Gestione_Recensioni.view	5	0% 0/77	0% 0/22	100% 0/0
sottosistemi.Gestione_Utenti.service	3	88% 111/126	89% 57/64	92% 57/62
sottosistemi.Gestione_Utenti.view	12	0% 0/300	0% 0/133	100% 0/0
utilities	2	38% 8/21	18% 3/17	60% 3/5

- **Baseline:** line coverage 59% (1,185/2,012), mutation coverage 47% (369/784), test strength 75% (369/494)
- **Interventions:** new tests for previously unconsidered combinations of conditions, more accurate verification of return values and side effects, tightened assertions.
- **Final results:** line coverage 99% (2,558/2,590), mutation coverage 98% (976/999), test strength 98% (976/997).

Mutation testing with PIT

AFTER

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
53	99% 2558/2590	98% 976/999	98% 976/997

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage	Test Strength
model	1	95% 20/21	100% 8/8	100% 8/8
model.DAO	10	99% 825/826	100% 298/298	100% 298/298
model.Entity	10	100% 245/245	100% 43/43	100% 43/43
sottosistemi.Gestione_Catalogo.service	1	100% 75/75	100% 44/44	100% 44/44
sottosistemi.Gestione_Catalogo.view	8	97% 365/378	95% 129/136	96% 129/134
sottosistemi.Gestione_Recensioni.service	1	100% 117/117	98% 83/85	98% 83/85
sottosistemi.Gestione_Recensioni.view	5	99% 202/205	96% 81/84	96% 81/84
sottosistemi.Gestione_Utenti.service	3	100% 141/141	96% 74/77	96% 74/77
sottosistemi.Gestione_Utenti.view	12	98% 547/561	96% 199/207	96% 199/207
utilities	2	100% 21/21	100% 17/17	100% 17/17

- **Baseline:** line coverage 59% (1,185/2,012), mutation coverage 47% (369/784), test strength 75% (369/494)
- **Interventions:** new tests for previously unconsidered combinations of conditions, more accurate verification of return values and side effects, tightened assertions.
- **Final results:** line coverage 99% (2,558/2,590), mutation coverage 98% (976/999), test strength 98% (976/997).

6

Performance testing

How we approached performance

- ▶ The platform had already been optimized for the Software Sustainability exam, but it grew with the features added for the Advanced Techniques exam, so the whole cycle was repeated on the current codebase.
- ▶ **Method:** measure first, change only what the measurement justifies, then measure again under identical conditions.



STEP 1 • MEASURE

JMH

Microbenchmarks on the service layer establish a repeatable baseline in ns/op.



STEP 2 • DIAGNOSE

Creedengo

Static analysis points at the code patterns that waste CPU, memory and database I/O.



STEP 3 • ACT

Impact-driven fixes

Findings are prioritized by their real cost, not by how often they occur.



STEP 4 • VERIFY

JMH re-run

The same benchmarks, the same configuration: the before / after gap is the evidence.

JMH microbenchmarks — measurement criteria

- ▶ **Configuration:** Mode.AverageTime, NANOSECONDS unit (ns/op), warmup 15 iterations of 1 s, measurement 40 iterations of 1 s, fork 2.
 - The 15 warmup iterations were selected after testing several combinations: the optimal value for stabilizing AverageTime, reducing variability from JIT compilation, profiling and caching.
- ▶ **Components measured:** CatalogoService, RecensioniService, AutenticationService, ProfileService, ModerationService, FieldValidator, PasswordUtility.
- ▶ **Constant operating conditions:** Lenovo laptop with AMD Ryzen 5 5500U, 8 GB RAM, Windows 11 (24H2), JDK 17, MySQL 9.2.0.
- ▶ **Traceability:** all results exported to txt and versioned in the repository, split into before and after directories.

15 + 40

warmup + measurement iterations

fork 2

independent JVMs per benchmark

7

benchmark suites versioned

Static analysis with Creedengo

▼ Rule

🔍

Search for rules...

(Java) Variable can be made constant	140
(Java) Don't use the query SELECT * FROM	18
(Java) Use ++i instead of i++	3
(Java) Avoid getting the size of the collection in t...	2
(Java) Do not call a function when declaring a for...	2
(Java) Avoid usage of static collections.	1

6 shown

- ▶ **166 violations across 6 categories:** Creedengo's goal is not style cleanup, but detecting patterns that introduce resource waste.
- ▶ **SELECT * FROM (18) — high priority:** the most critical violation; retrieves unnecessary columns, inflating disk I/O, network transfer and memory consumption.
 - **Fix:** queries rewritten listing only the required fields explicitly.
- ▶ **Static collections (1) — high priority:** persist in memory throughout the entire application lifecycle.
 - **Fix:** converted into an instance/local variable, memory released by the GC at the end of the request.
- ▶ **Loop micro-optimizations (7) — high priority:** ++i instead of i++, collection size evaluated in the loop condition, function calls in the for declaration.
 - **Fix:** invariant calls and .size() extracted outside the loop, computed once and reused.
- ▶ **Variables that can be final (140) — low priority:** 84% of the cases; immediate impact limited by JIT optimizations, but it affects the overall energy footprint. Resolved with the IDE's automated refactoring.

Validation: before / after comparison

Static analysis re-run with the same Quality Profile: total resolution of all violations tagged with Creedengo.



ProfileService benchmark	Before (ns/op)	After (ns/op)	Reduction
benchmarkAggiungiAllaWatchlist	4,540.797	3,607.234	-20.56%
benchmarkPasswordUpdate	9,982.091	7,716.780	-22.69%
benchmarkProfileUpdate	14,696.992	11,558.612	-21.35%

Excerpt: the repository contains the full suite of measurements for the other services (JMH/before and JMH/after directories).



Security

Native GitHub security: shift left model

Goal: identify and mitigate security risks at the earliest possible stage, stopping vulnerabilities before the code is ever released.

Supply Chain Security

Dependency Graph + Dependabot

Project dependencies are analyzed automatically and continuously cross-referenced against up-to-date vulnerability databases.

Dependabot generates corrective pull requests, also in grouped format: this reduces the likelihood of incorporating compromised libraries and accelerates response to external security events.

Secret Protection

Secret Scanning + Push Protection

Proactive scanning for patterns matching sensitive credentials: tokens, API keys and private secrets.

Push protection acts as a preventive gatekeeper, blocking pushes when secrets are detected: credentials never enter the git history, avoiding revocation and rotation procedures.

Vulnerability Reporting

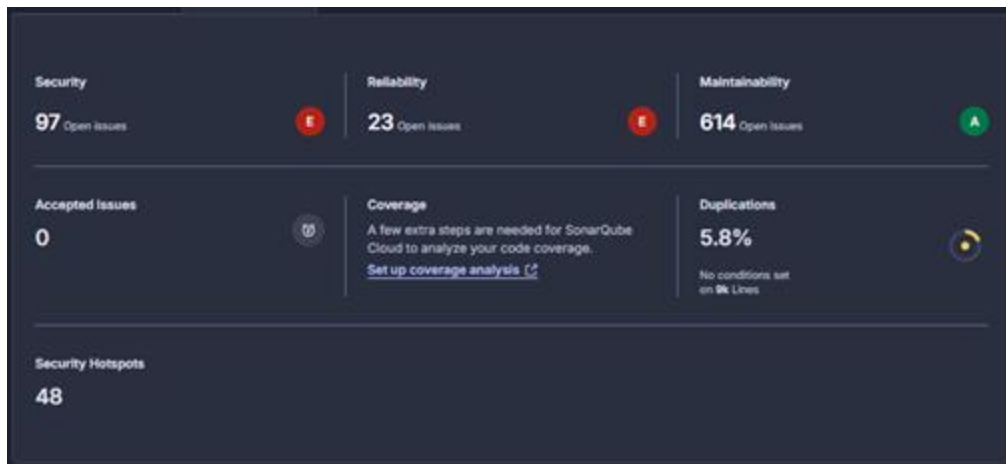
Private Vulnerability Reporting

A dedicated, secure channel lets users and researchers privately report potential security concerns.

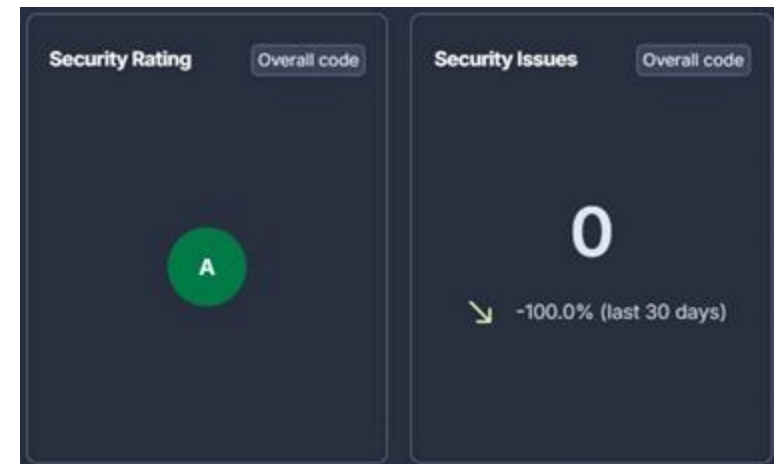
It establishes a responsible disclosure framework: the team can analyze and resolve vulnerabilities confidentially before public disclosure.

SonarQube: from 97 to 0 security issues

- ▶ **Explicit, measurable objective:** zero security vulnerabilities across all adopted tools.
- ▶ **Single recurring root cause:** forward calls executed without explicitly handling ServletException and IOException.
- ▶ **Potential impact:** flow interruptions, inconsistent state responses or accidental exposure of detailed error stack traces, compromising overall robustness.
- ▶ **Applied fix:** all forward calls encapsulated within try/catch blocks, so errors are handled in a controlled and secure manner.



before



after

GitGuardian — secret detection

4 findings raised on the repository, all of them **manually reviewed** and confirmed as **false positives**.

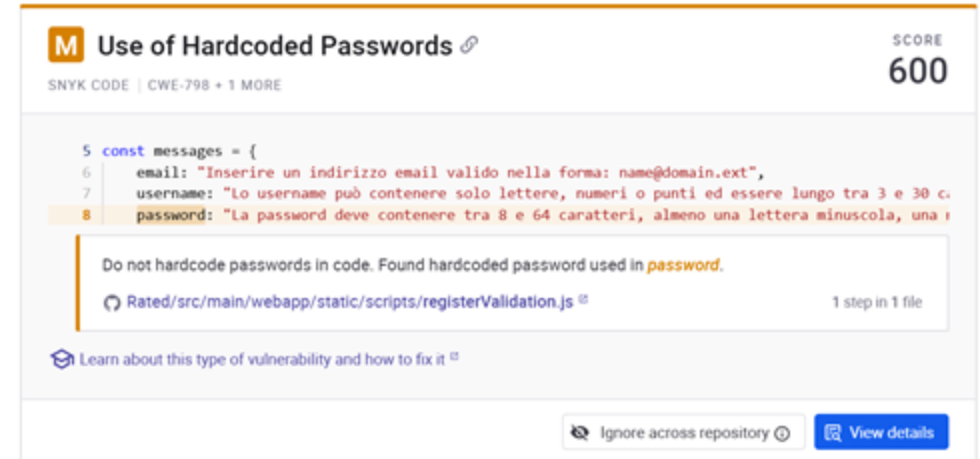
☐	Occurred date	Validity ⓘ	Secret	Severity	Info	Tags	Status
☐	Jul 14th 23:58	No checker	Username Password 🔍 File 1	High	superfra01/Rated-Dependability Rated/src/test/java/unit/test_model/UtenteBeanTest... superfra01 151184247+superfra01@users.noreply.github.com	Test file Default branch	Ignored Not a secret (false positive) ⓘ
☐	Jul 14th 23:58	No checker	Generic Password 🔍 File 1	High	superfra01/Rated-Dependability Rated/src/test/java/unit/test_model/FieldValidatorTe... superfra01 151184247+superfra01@users.noreply.github.com	Test file Default branch	Ignored Not a secret (false positive) ⓘ
☐	Jul 14th 15:08	No checker	Generic Password 🔍 File 3	High	superfra01/Rated-Dependability Rated/docker-compose.yml superfra01 151184247+superfra01@users.noreply.github.com	Default branch	Ignored Not a secret (false positive) ⓘ
☐	Feb 23rd 18:02	No checker	Generic Password 🔍 File 4	High	superfra01/Rated-Dependability Rated/benchmark/AuthenticationServiceBenchmark.j... superfra01 +1 151184247+superfra01@users.noreply.github.com	From historical scan Default branch	Ignored Not a secret (false positive) ⓘ

GitGuardian scan results: every finding marked as "Not a secret (false positive)"

- **Findings 1–2:** dummy credentials used strictly for testing activities.
- **Finding 3:** it is Bash and Docker Compose parameter expansion, which halts execution and prints the message if DB_PASSWORD is null or unset. It is in fact a validation check, flagged only because the scanner detected the keyword PASSWORD.
- **Finding 4:** a benchmarking script, with no security risk.

Snyk Code: static application security testing

- ▶ **Objective:** zero issues, eliminating any potentially vulnerable pattern identified in the initial run.
- ▶ **Result:** a single finding, manually verified as a false positive and the scanner detected the word "password" in a non-sensitive context.
 - The match is on the client-side validation messages of the registration form (registerValidation.js): no secret is exposed and no corrective action was required.
- ▶ **Snyk in the CI pipeline:** Open Source (SCA) on Maven dependencies with a strict minimum severity threshold, Monitor tracking dependency health on the main branch, Code (SAST) honouring the exclusions declared in the .snyk file.



Snyk Code — the only reported finding

8

Final considerations

Final considerations

- ▶ Trough exhaustive tools use, a structured CI/CD pipeline, a rigorous focus on code quality and embedded automated security checks, we made the Rated project more dependable and reliable to the users.
- ▶ Every optimized metric, resolved vulnerability and automated validation step reinforces the application's stability, even when handling edge cases: the Rated architecture is now significantly more robust and maintainable.

96% / 93%

instructions / branches
(JaCoCo)

98%

mutation coverage (PIT)

0

exit code of OpenJML ESC

-22.7%

ns/op on the best
benchmark

0

security vulnerabilities



RATED

•COMMUNITY REVIEWS•

Thanks for your attention!